

Generative Adversarial Network (GAN) on MNIST Dataset

1. Objective

The objective of this project is to implement and train a Generative Adversarial Network (GAN) using TensorFlow (Keras API) to generate realistic handwritten digit images similar to the MNIST dataset.

GANs consist of two neural networks:

- **Generator** – generates fake images from random noise
- **Discriminator** – distinguishes between real and fake images

The goal is to train both models in a competitive setting such that the generator produces highly realistic images.

2. Brief Explanation of GANs

A GAN is a deep learning framework introduced to generate new data samples from a learned distribution.

- The Generator takes a random noise vector (100-dimensional) and converts it into a 28×28 grayscale image.
- The Discriminator evaluates whether an image is real (from dataset) or fake (from generator).

Working Principle:

- Generator tries to fool the discriminator
- Discriminator tries to detect fake images
- Both improve through adversarial training

This process continues until the generated images become indistinguishable from real ones.

3. Dataset and Preprocessing

- Dataset used: MNIST handwritten digits
- Total images: 60,000
- Image size: 28×28 grayscale

Preprocessing steps:

- Normalized pixel values to range $[-1, 1]$
 - Reshaped images to $(28, 28, 1)$
 - Batched and shuffled using TensorFlow dataset API
-

4. Architecture Summary

Generator Network

- Input: 100-dimensional random noise vector
- Fully connected layer $\rightarrow$ reshaped to $7 \times 7 \times 256$
- Upsampling using Conv2DTranspose layers
- Activation: LeakyReLU
- Output layer uses tanh activation
- Output: $28 \times 28 \times 1$ image

Discriminator Network

- Input: 28×28 image
 - Convolutional layers with stride for downsampling
 - Activation: LeakyReLU
 - Dropout layers for regularization
 - Fully connected layer $\rightarrow$ single output (real/fake)
-

5. Loss Functions

- Binary Cross-Entropy Loss is used

Generator Loss:

- Measures how well generator fools the discriminator

Discriminator Loss:

- Measures ability to distinguish real vs fake images

6. Results

Loss Graphs

- Generator loss initially fluctuates and then stabilizes
- Discriminator loss increases slightly over time

This indicates:

- Generator is improving
- Discriminator becomes less confident

Generated Outputs

- Images generated after each epoch show gradual improvement
- Early epochs: noisy and unclear digits
- Later epochs: clearer and more structured digits

7. Observations

- GAN training is unstable and sensitive
- Generator and discriminator must be balanced
- Overpowering one network leads to poor results
- Image quality improves gradually with epochs

8. Challenges Faced

1. Training Instability
 - Loss values fluctuate significantly
 - Hard to determine convergence
2. Slow Training
 - Training takes time without GPU

- Requires optimization (batch size, epochs)
 - 3. Mode Collapse
 - Generator may produce similar images repeatedly
 - 4. Balancing Networks
 - If discriminator becomes too strong, generator fails
 - If generator dominates, discriminator becomes weak
-

10. Conclusion

The GAN model successfully generated handwritten digit images resembling the MNIST dataset. Although training was challenging due to instability, techniques like label smoothing and balanced training improved performance.

GANs are powerful models for generative tasks, but require careful tuning and computational resources.